Developer Guide

TGIR QuantLib Tools

TG Investments and Research

2026-04-06

Contents

1 Audience And Goal

This guide is for developers who need to understand the application structure, local workflow, extension points, and delivery expectations.

2 Repository Layout

Path	Responsibility
`app.py`	Local Flask entrypoint.
`portfolio.py`	QuantLib market construction, trade pricing, and analytics logic.
`tgir_quantlib_tools/app_factory.py`	Flask app construction.
`tgir_quantlib_tools/routes.py`	Route registration and HTTP handlers.
`tgir_quantlib_tools/dashboard.py`	Session state, dashboard payload shaping, and trade-editor context building.
`tgir_quantlib_tools/quantlib_model.py`	Data-model page and research section context.
`tgir_quantlib_tools/research.py`	Curated paper metadata for swaption and cliquet references.
`templates/`	Jinja templates for the UI.
`tests/`	Smoke, calibration, and identity tests.
`docs/latex/`	Audience-specific LaTeX documentation.

3 Application Flow

1. `create_app()` loads configuration and builds the Flask app.
2. Authentication and template filters are registered.
3. Requests hit `tgir_quantlib_tools/routes.py`.
4. Protected routes load session state from `tgir_quantlib_tools/dashboard.py`.
5. Pricing functions in `portfolio.py` normalize state and construct QuantLib objects.
6. Templates render the final dashboard, editor, or model/research page.

4 State Model

The session state is a nested dictionary with:

- `market` for rates, swaption volatility, shared mean reversion, and SPX equity assumptions,
- `trades.swap`,
- `trades.european_swaption`,
- `trades.bermudan_swaption`, and

- `trades.bermudan_swaption_2`, and
- `trades.equity_cliquet`.

Always pass data through `normalize_portfolio_state()` before constructing pricing objects.

5 Pricing Responsibilities

5.1 portfolio.py

This is the core quantitative module. It owns:

- curve normalization and construction,
- swaption-volatility matrix normalization,
- swap, swaption, and cliquet instrument creation,
- European Hull-White 1F + Jamshidian pricing,
- Bermudan Hull-White 1F / G2++ calibration and tree pricing,
- price tables,
- risk ladders, and
- cliquet scenario and Monte Carlo analytics.

5.2 dashboard.py

This module should stay presentation-oriented. It shapes payloads, form definitions, and display-specific summaries, but should not duplicate pricing logic already available in `portfolio.py`.

6 How To Add Or Change A Trade

1. Extend `default_portfolio_state()` and normalization rules.
2. Add a human-readable entry in `TRADE_SEQUENCE` and `TRADE_TITLES`.
3. Implement instrument creation and pricing in `portfolio.py`.
4. Add form definitions and update handlers in `tgir_quantlib_tools/dashboard.py`.
5. Add template support in `templates/dashboard.html` and `templates/trade\_form.html`.
6. Extend smoke and regression tests before merging.

7 Local Development Workflow

7.1 Setup

```
python3 -m venv .venv
./.venv/bin/python -m pip install -r requirements.txt
cp .env.example .env
```

7.2 Useful Commands

```
./.venv/bin/python app.py
./.venv/bin/python build_SOFR_curve.py
./.venv/bin/python -m unittest discover -s tests
./.venv/bin/python -m py_compile portfolio.py tgir_quantlib_tools/*.py
```

8 Documentation Workflow

The LaTeX docs are built with:

```
make -C docs/latex
```

Keep audience-specific material separated:

- end users: workflows and UI,
- quants: models and assumptions,
- developers: architecture and extension points,
- IT: runtime operations,
- testing: regression methodology,
- deployment: CI/CD and DigitalOcean procedures.

9 Testing Expectations

Changes that touch pricing must update tests in one of three layers:

- `tests/test\_sofr\_curve.py` for calibration or rates logic,
- `tests/test\_cliquet.py` for cliquet payoff identities, and
- `tests/test\_smoke.py` for route and rendering behavior.

10 Change Review Checklist

- Did the change preserve state normalization?
- Is new pricing logic in `portfolio.py` rather than duplicated in templates or routes?
- Do calibration instruments still reprice?
- Are tests deterministic?
- Are docs updated in both Markdown and LaTeX where relevant?